SPACESHIP MANAGEMENT GAME — VISION MANIFESTO

Status: FINAL DESTINATION DOCUMENT. Not a Phase 1 spec.
Note: All examples within are ILLUSTRATIONS, NOT LIMITS.
Anchor to the principles behind the examples.

SECTION 1: THE CORE IDENTITY

1.1 What This Game Is

This is a spaceship management game. The player is the commander of a vessel traveling through space. They manage everything — the ship's systems, the crew, the resources, the missions, the finances, and eventually much more.

But this is not a game about flying a ship. There is no joystick. No cockpit view. No space battles the player fights manually. This is a game about managing information and making decisions. The player reads, observes, clicks, assigns, repairs, trades, hires, fires, investigates, responds to crises, and slowly builds their operation from almost nothing into something vast and complex.

The entire game is presented as a dashboard. Think of it as the software interface that a real spaceship commander would sit in front of. Panels of information. Status readouts. Crew manifests. System diagnostics. Task queues. Warning indicators. Everything the player needs to run their ship is presented as clean, readable, clickable information — like a website, like an internal company tool, like a hospital management system, but for a spaceship.

The game starts extremely small. The player begins with a tiny ship. Three crew members. A handful of rooms. Very few systems to worry about. Very few things to click on. This is intentional. The player is not told how to play through a tutorial popup. They learn by doing. They see a crew member dot on the ship. They right-click it. A task opens. They see what they can do. They click something. Something happens. They learn. The game teaches through interaction, not instruction.

Over time — and this is important — the game grows. Not all at once. Not on a schedule. But as the player progresses, earns Shiplets (the in-game currency), docks at space stations, completes quests, and makes decisions, new things become available. A bigger ship. More crew. New ship systems. New room types. New problems. New event chains. New things appearing on the idle screen that weren't there before. Systems that didn't exist at the start now need attention. Crew members who were content now have complaints. Visitors arrive with demands. The quiet, peaceful early game slowly transforms into a dense, layered, demanding management experience — but the player was eased into it so gradually that they are ready for it.

This growth is not just "more content." It is a fundamental expansion of complexity. Early game, the player might have 3 tasks open and each costs 1-2 actions. Late game, the player might have 15 tasks open, each requiring 10-50 actions, with dozens of countdowns ticking simultaneously, with

event chains triggering automatically in the background while the player is busy handling something else. The game does not get harder by making numbers bigger. It gets harder by making the web of responsibilities denser.

1.2 What This Game Is NOT

It is not real-time. Nothing moves forward on a clock. If the player walks away from the screen, nothing changes. If the player stares at the idle screen for an hour, nothing progresses. Time only advances through actions, and actions only happen inside task windows. This is a core rule that must never be broken.

It is not a clicker game. Actions have weight. Each click inside a task window is a deliberate decision. The player is not mindlessly clicking to fill a bar. They are choosing what to do, when to do it, and in what order — knowing that every action advances countdowns on other things they may not be paying attention to.

It is not a visual spectacle. There are no particle effects, no cinematic cutscenes, no elaborate animations. The ship cross-section on the idle screen has dots moving around — that is the most "animated" the game gets. Everything else is information. Text. Numbers. Progress bars. Status indicators. Buttons. Lists. Tables. The beauty of the interface comes from its clarity and organization, not from visual flair.

It is not a sandbox without consequence. While there is no win condition, the player can lose. Systems can fail. Crew can die. The ship can be destroyed. Money can run out. Bad decisions have real outcomes. The player must care about what they are doing.

It is not a game that dumps everything on the player at once. This cannot be overstated. The early game must feel almost empty. Almost too simple. The player should wonder "is this it?" for a little while — and then slowly realize that the game has been quietly expanding around them. A new indicator appeared on the sidebar. A crew member has a new option available. A quest offers something that wasn't possible before. The growth is organic, not sudden.

1.3 The Action Economy

This is the central mechanic of the entire game. Everything revolves around it. If the agent does not understand this system deeply, the game will not work. So this section must be very clear.

The idle screen is free. The player can look at the ship cross-section, watch dots move, read the stats around the hull, check the sidebar indicators, hover over things, observe — all of this costs nothing. No actions are consumed. No countdowns advance. The idle screen is a safe harbor. It is the place where the player breathes, thinks, and plans.

Task windows cost actions. When the player right-clicks on something (a crew member, a room, a system, the docked station symbol, anything interactive on the idle screen), a task opens in the top taskbar. This task appears as a tab, like a browser tab. Clicking that tab takes the player into the task window. Inside the task window, the player sees information about that thing and is presented with buttons, choices, and interactions. Every interaction inside a task window is an action. Every action advances the game's internal action counter.

The action counter drives everything. When the action counter advances by 1, every active

countdown in the game ticks down by 1. Every running event chain checks if it should advance. Every pending outcome checks if it should trigger. The action counter is the heartbeat of the game — and the player controls how fast it beats.

Not all actions cost the same. Some actions cost 1 click. Talking to a crew member might be 1 action. Washing in the bathroom might be 1 action. But some actions are multi-step processes. Repairing the engine might take 20 actions. Traveling to a location might take 10 actions. These are not 10 separate button presses — the player initiates the action once, and then that process is "running" and will complete after the player has performed 10 total actions anywhere else in the game. The player starts the engine repair (1 click to initiate, but it takes 20 more actions anywhere in the game to complete). While it runs, they can go do other things — talk to crew, manage cargo, handle a quest — and each of those clicks also advances the repair countdown.

This creates the core tension. The player wants to do many things but every action they take advances every timer simultaneously. Early game, this is invisible — there are so few timers that the player doesn't notice. Late game, this is the entire strategic challenge. The player must prioritize: "If I spend 15 actions handling this crew dispute, my shield repair will complete but my hull integrity event chain will advance to a critical stage and I might not have time to address it."

The trick — the real trick — is that the player must eventually be forced to act. Early game, the player can idle on the main screen as long as they want. But as the game grows, situations arise where the player cannot stay on the idle screen. A critical event demands attention. A countdown is almost at zero. A crew member is about to snap. The player must enter a task window and spend actions — knowing full well that those actions advance other countdowns they're worried about. The idle screen transforms from a peaceful home base into a brief pause between crises.

But even this builds gradually. At the start, the player might go 50 actions before anything urgent happens. Later, something is always urgent. This curve must be gentle and natural, never a sudden spike.

1.4 Events

Events are what bring the game to life. Without the event system, the game is just a dashboard of static numbers. Events are changes, surprises, consequences, chain reactions, crises, opportunities, and stories.

Events are not simple popups that say "a thing happened, click OK." Events are systemic. They have causes. They have consequences. They interact with each other. They can trigger other events. They can require the player to respond across multiple actions. They can resolve themselves without player input. They can escalate if ignored. They are the living tissue of the game.

Here are the types of events the game must support. These are categories, not a complete list. The game will eventually have far more event behaviors than these examples suggest. The examples exist to illustrate the category, not to limit it:

Immediate events: The player performs an action, and something happens right now. No delay. No countdown. The player clicks "assign crew member to engine room" and the crew member is now in the engine room. The player clicks "eat meal" and the crew member's hunger decreases. Simple cause-and-effect.

Delayed events: The player performs an action, and the outcome will happen after a certain number

of actions have passed. The player clicks "begin shield repair" and the repair will complete after 20 more actions are taken anywhere in the game. During those 20 actions, the shield repair is "in progress" and visible as a countdown somewhere. The player does not need to babysit it — they just need to keep playing and it will resolve.

Automatic chain events: Something triggers a chain of events that will play out on its own, without the player doing anything specific. For example: a hull micro-fracture is detected -> 5 actions later, the fracture spreads -> 8 actions later, atmosphere begins leaking in that section -> 3 actions later, crew in that section begin evacuating on their own -> the section is sealed automatically. The player watches this happen through indicators and status changes. They may or may not be able to intervene — but the chain runs regardless. These chains give the ship a sense of being alive. Things happen aboard. The crew reacts. Systems respond. The player is the commander, not the micromanager of every molecule.

Player-driven chain events: Similar to automatic chains, but each step in the chain requires the player to make a choice or take an action before the next step triggers. For example: the player initiates a diplomatic negotiation -> after 3 actions, the diplomat presents a demand -> the player must respond (accept, reject, counter-offer) -> based on the response, 5 actions later another event fires -> the player responds again -> and so on. These chains are conversations, negotiations, multi-step repairs, investigations, or any process where the player must be actively involved across multiple decisions spread over many actions.

Sequence-gated events: Some events only trigger if the player has performed a specific sequence of actions. Not just "did action A happen" but "did action A happen, then action B, then action C, in that order, and maybe within a certain action window." These are discoverable by the player through experimentation and observation. They are not explained in advance. The player figures out patterns. "Oh, every time I clean the engine room, then check the fuel lines, then run diagnostics, I get a bonus efficiency boost. But if I skip the cleaning step, it doesn't trigger." This rewards attentive, thoughtful play and gives the game hidden depth.

Persistent events: Some events last over many actions. They are not a single moment — they are a condition that persists. A solar storm that lasts for 30 actions, during which all external repairs take twice as long. A diplomatic visit that lasts for 15 actions, during which the visitor's mood is tracked and affected by the player's other actions. A disease outbreak that persists until the player completes a multi-step medical procedure. These events create ongoing pressure and context that colors everything else the player is doing.

Cascading events: Events that trigger other events, which trigger other events. A power surge damages the engine -> the engine shutdown triggers life support to switch to backup -> backup power drain triggers a warning about oxygen reserves -> oxygen warning triggers crew anxiety increase -> crew anxiety triggers a fight between two crew members -> the fight triggers a medical emergency. The player may have started this cascade with a single careless action 40 actions ago. Or it may have started from an automatic event they didn't notice. Cascading events are what make the late game feel overwhelming and alive. They are what punish neglect and reward attentiveness.

Hidden events: Some events happen without the player being explicitly told. A crew member's mood drops slightly. A system degrades by 1%. A visitor forms a negative opinion. These are not announced with popups or warnings. They happen quietly, reflected only in the numbers on the dashboard — if the player is paying attention. Hidden events reward the player who reads their dashboard carefully and notices small changes. They punish the player who only reacts to loud warnings.

The event system is the most important system in the game from an architecture perspective. It must be built to be easily expandable. Adding a new event type, a new chain, a new trigger condition, a new consequence — all of these must be straightforward for the developer (or the agent in future sessions) to add without rewriting the core engine. Events should be defined as data wherever possible — objects with properties like trigger conditions, action costs, countdown lengths, outcomes, chain links — not hard-coded logic buried in functions.

1.5 The Visual Design

The game looks like a clean, well-organized website. Not a futuristic holographic display. Not a retro green-on-black terminal. Not a sleek modern web application with rounded corners and gradients everywhere. Just... normal. Like a competent internal tool built by someone who values readability over style.

Color palette:

Primary background: white. Clean, bright white.
Secondary backgrounds (panels, cards, sections): very light warm grays, off-whites, creams.
Text: dark gray or soft black. Never harsh pure black on pure white — slightly softened.
Accent color: coffee. Warm brown tones. Think latte, cappuccino, tan, light brown. Used for borders, highlights, active states, selected tabs, important indicators.
Warning/danger: muted warm red. Not fire-engine red. More like a brick or terracotta.
Success/positive: muted warm green. Not neon. More like sage or olive.
Neutral indicators: the coffee brown tones.

Typography:

Clean, readable, standard. No monospace unless displaying data/numbers. No decorative fonts. No sci-fi fonts. Just a good, normal, readable sans-serif font that comes standard with the operating system.

Layout philosophy:

Information density is high but organized. The player should see a lot on screen, but it should never feel cluttered. Proper spacing. Proper alignment. Proper grouping. Related information is visually grouped. Unrelated information is visually separated.
No unnecessary decoration. No background images. No textures. No gradients (except very subtle ones if needed for depth). No shadows (except very subtle ones to separate layers). Function over form, always.
Responsive to window size where possible, but the primary design target is a comfortable desktop window — not mobile, not fullscreen necessarily.

The idle screen (ship cross-section):

Center of the screen. The largest visual element.
Shows the ship from the side, cut open so all interior rooms/decks are visible. Like looking at a dollhouse, or an ant farm, or a submarine cross-section in a technical manual.
Rooms are simple rectangular areas with labels. Engine room. Bridge. Medical bay. Crew quarters. Cargo hold. Each room is visually distinct but simple — maybe different subtle background colors or small icons.
Dots move within the ship. Each dot represents a person — crew member, worker, visitor. Dots

move between rooms slowly. They pause in rooms (working, resting, socializing). The movement is not animated in real-time — remember, nothing happens in real-time. The dots reposition each time the action counter advances. Between actions, they are still. But because the player spends time on the idle screen between actions, they see dots in their current positions, giving a sense of life.
Wait — let me reconsider. The dots DO move on the idle screen. Even though the game is action-based, the idle screen is the one place where there is gentle ambient movement. Dots drift along slowly. Not because time is passing — but because the idle screen is a representation, a visualization, a living diagram. The dots moving is cosmetic. It shows "this is where people are and what they're generally doing." It does not advance the game state. It is purely visual life. This is important for making the idle screen feel alive rather than frozen.
Around the exterior of the ship diagram: small stat displays. Hull integrity. Shield status. Energy level. Fuel. Nearby objects. Current location. These are always visible. All of them are right-clickable to open tasks.
Below or beside the ship: possibly a small event log or recent activity feed. Short lines of text: "Crew member Jax entered the engine room." "Shield generator operating at 78%." "Docked at Station Velmora." This feed updates as actions are taken and events fire. It is scrollable. It is informational. It is not intrusive.

The top taskbar:

Runs along the top of the window.
Contains tabs. Each tab represents an open task.
The player can have many tabs open simultaneously.
Clicking a tab switches the main content area from the idle screen to that task's window.
There should always be a way to return to the idle screen (a "home" tab, or clicking outside tasks, or a dedicated button — the exact UX can be decided, but the path back to idle must always be obvious and easy).
Tabs can be closed when the player is done with a task (or when a task resolves itself).
Tabs might have small indicators on them — a dot, a color change, a number — to show that something has changed in that task since the player last looked. "Your engine repair completed while you were in the crew management task. The engine tab now has a green dot."

The side panel:

Runs along one side of the window (left or right — to be decided).
Contains general stats, ship status, resource counts, and — critically — the cryptic countdown indicators.
The countdown indicators are deliberately vague. They do not say "hull breach in 7 actions." They might show a small icon with a color that shifts from green to yellow to orange to red. Or a progress bar that fills up ominously without a label. Or a symbol that the player learns to recognize through experience. The player must learn what these indicators mean through play. Some indicate countdowns. Some indicate event chain progress. Some indicate crew moods. Some might indicate things the player hasn't even discovered yet.
The side panel also contains buttons or links for general game functions: save game, load game, settings (if any), and perhaps general reference information like a ship manifest or a log.
Items
in the side panel may open as popups or floating windows rather than taking over the main content area. This way the player can keep the idle screen visible while checking a stat popup.

1.6 The Progression Arc

Starting state:

A small ship. Very small. Maybe 4-5 rooms: bridge, engine room, crew quarters (shared), a small cargo hold, and maybe one utility room.
3 crew members. Each has a name, a role, basic needs (sleep, food, morale), and a small set of things the player can interact with.
Very few active systems. Power works. Life support works. Maybe shields exist but are basic.
The player's first quest is simple. Travel to location X. This costs maybe 10 actions (initiated once, then the travel "runs" for 10 actions). During travel, the player can explore the other task options — check on crew, look at ship systems, explore what right-clicking things does.
Money: the player starts with a small amount of Shiplets. Enough to not worry about money yet.

Early game:

Simple quests: deliver cargo, transport a passenger, scan an anomaly.
Rewards: Shiplets, maybe a small piece of equipment.
The player learns the action economy naturally. They click things, stuff happens, they start noticing the counters.
Docking at space stations: the player encounters station tasks. Buy supplies. Hire a fourth crew member (eventually). Browse upgrades they can't afford yet. Visit the tavern (maybe hear rumors that are actually quest hooks).
Very few events fire. Maybe one small event chain every 30-50 actions. A crew member gets bored. A minor system needs maintenance. Nothing urgent. Nothing scary.

Mid game:

The ship is bigger. Maybe upgraded once or twice, or the player bought a new ship.
6-10 crew members. They have relationships with each other. Specializations matter more. Some tasks require specific crew skills.
Ship systems are more numerous and some require regular maintenance.
Quests are longer and more complex. Multi-step quest chains. Choices that affect outcomes.
Events fire more frequently. Every 10-20 actions, something demands attention.
The player has many tabs open. They switch between them, prioritizing.
Visitors start appearing. Traders, diplomats, passengers. Each brings their own task chains.
The player starts feeling the action economy's pressure. "If I deal with this trade negotiation (15 actions), my engine maintenance countdown will hit zero and I'll have a problem."

Late game:

Large ship (or fleet? maybe eventually). Many rooms. Many systems. Many crew.
Events are frequent and overlapping. Cascading chains. Persistent conditions.
The sidebar indicators are dense with cryptic warnings.
The idle screen is alive with dots and information.
The player has deep knowledge of the game's patterns. They recognize what the cryptic indicators mean. They know which sequence-gated events to trigger for bonuses. They plan ahead.
But the complexity is always threatening to overwhelm. A new event type they haven't seen before appears. A cascade they didn't predict. A crew crisis at the worst possible moment.
The game doesn't end. It keeps going. The player manages, adapts, survives, grows.

1.7 Currency and Economy

Shiplets are the universal currency. They are earned through:

Completing quests
Trading goods at space stations
Selling salvage
Passenger transport fees
And eventually many other sources as the game expands

Shiplets are spent on:

Ship upgrades and new ships
Hiring crew members
Buying supplies (food, fuel, repair materials, medical supplies)
Buying equipment and tools
Docking fees at stations
And eventually many other costs

The economy should feel tight but not suffocating in the early game, and flexible but demanding in the late game. The player should never feel like they have infinite money. They should always have to choose: "Do I upgrade my shields or hire a new crew member? I can't afford both right now."

1.8 Space Stations and Docking

When the player's ship arrives at a space station (after a travel action countdown completes), a docked indicator appears on the idle screen. This is a visual symbol — an icon, a status text, something clear — that tells the player "you are docked."

Right-clicking the docked indicator opens the space station task in the top taskbar. This task window is the player's gateway to everything the station offers:

Market/shop: Buy and sell goods, equipment, ship parts, supplies. Prices may vary by station.
Crew hiring: Browse available crew members. Each has stats, skills, personality traits, and a hiring cost. The player can interview them (1-2 actions) before deciding.
Shipyard: View available ship upgrades or entirely new ships. Expensive. The player works toward these over time.
Tavern/social area: Hear rumors (quest hooks). Meet interesting characters. Maybe trigger special event chains. This is where flavor and story live.
Mission board: Pick up new quests. Some are simple fetch quests. Some are long, complex, multi-step operations.
Repair and maintenance services: Pay to have the station fix things instead of the player managing repairs manually. Costs more Shiplets but saves actions.

Each of these sub-areas within the station task window has its own interactions, all of which cost actions. The player must be deliberate about what they do at the station. Browsing is not free once they're inside the task window. "Do I spend 5 actions interviewing crew candidates, or do I use those actions to negotiate a better trade deal?"

The player can undock and leave at any time (1 action). Travel to the next destination begins.

Not all stations are the same. Some are large with many options. Some are small outposts with limited supplies and no tavern. Some might be hostile or have special conditions. This variety is

part of the game's expandability — new station types can be added over time.

1.9 Crew

Crew members are the heart of the game. They are not abstract resources. Each crew member is an individual with:

A name. Generated or chosen. They are a person.
A role/specialty. Engineer, medic, pilot, security, scientist, cook, etc. Their role determines what tasks they perform well and what stations on the ship they are assigned to.
Stats. These can include but are not limited to: skill level, experience, health, hunger, fatigue, morale, stress. Stats change over time (action time) based on what happens.
Needs. Crew members need to eat, sleep, socialize, and feel useful. If needs are neglected, stats degrade, moods worsen, events trigger.
Personality traits. Some crew members are calm under pressure. Some are prone to arguments. Some are secretly brilliant. Some are lazy. Some are loyal. Some might betray you. Traits affect how they behave during events and interact with other crew members.
Relationships. Crew members have opinions of each other. They form friendships, rivalries, romances, grudges. These relationships affect morale, cooperation, and trigger events. Two crew members who hate each other assigned to the same room? Expect problems.
A story. Over time, each crew member accumulates a history within the game. Tasks they completed. Events they were involved in. Injuries they suffered. Promotions they earned. The player develops attachment to their crew — or resentment.

Right-clicking a crew member's dot on the idle screen opens their personal task window in the top taskbar. Inside this window, the player can:

View all their stats and current status
See their current location and activity
Assign them to a room or task
Talk to them (costs 1 action — may reveal information, improve morale, trigger events)
Promote or demote them
Address issues (medical problems, complaints, conflicts)
Fire them (with consequences — other crew may react)
And eventually much more as the game expands

Crew members are visible on the idle screen as dots. Their dot might have a tiny color or shape indicator showing their role or current mood. They move between rooms. They do their jobs. They go to the mess hall to eat. They go to their quarters to sleep. They visit each other. All of this is ambient — it happens on the idle screen as cosmetic animation. But the actual state changes (hunger decreasing, fatigue building) only happen when the action counter advances.

Starting crew (3 members):

One should be decent at piloting/navigation
One should be decent at engineering/repairs
One should be decent at general duties
None of them are experts. They are a small, scrappy starting crew. The player grows fond of them or replaces them over time.

1.10 The Ship

The ship is both the player's home and their primary management challenge.

Physical layout:

Represented as a side cross-section on the idle screen.
Divided into rooms. Each room has a purpose.
Starting ship rooms might include: Bridge, Engine Room, Crew Quarters, Cargo Hold, Mess Hall (or a combined utility room).
As the player upgrades or buys new ships, more rooms become available: Medical Bay, Science Lab, Workshop, Armory, Guest Quarters, Recreation Room, Observation Deck, and many more.
Rooms can be upgraded too. A basic engine room becomes an advanced engine room. A cramped crew quarters becomes comfortable quarters with individual bunks.

Ship systems:

Each room is tied to one or more ship systems.
Life Support: Oxygen, temperature, atmosphere recycling. If this fails, crew start dying. Tied to a dedicated system room or the engine room.
Energy/Power: Powers everything. If energy runs low, systems start shutting down in priority order. The player may need to choose what to power down.
Shields: Protect the ship from external threats. Degrade over time or from damage. Need maintenance and energy.
Engines/Propulsion: Required for travel. Fuel consumption. Can break down.
Hull Integrity: The physical structure of the ship. Damage from events, collisions, or neglect. If it hits zero, catastrophic failure.
Communications: Contact
with stations, other ships, quest givers. If damaged, the player loses access to some information.
Navigation: Plotting courses, detecting nearby objects, avoiding hazards.
And more, over time. Weapons. Tractor beams. Scanners. Cloaking. Drones. The list grows as the game grows.

Each system has a status (operational, degraded, damaged, critical, offline) and a condition percentage. Systems degrade slowly through use (every X actions, a small degradation). They can be damaged by events. They can be repaired through crew actions (which cost actions from the action economy). They can be upgraded at stations.

The ship is a living machine. It is not a static backdrop. It reacts to events. Systems interact. If power drops below a threshold, life support switches to emergency mode. If the hull is breached in a section, that section is sealed and crew are rerouted. The ship has its own cascading logic that plays out through the event system.

1.11 Save and Load

The game saves to a .txt file on the player's local machine. This is possible because HTA files have access to the local filesystem through ActiveX/FileSystemObject.

What is saved:

Everything. The entire game state. Ship layout. Ship system statuses. Crew member data (stats,

location, relationships, history). Open tasks. Active events and their progress. Action counter. Shiplets balance. Current location. Quest progress. Discovered patterns. Everything.

Save format:

The .txt file will contain structured data. Most likely JSON-formatted text inside the .txt file. This makes it human-readable (for debugging or curiosity) and easy to parse back into the game.

When saving happens:

Manual save. The player clicks "Save" in the side panel. The game writes the current state to the file.
Optionally, auto-save every X actions as a safety net.

Loading:

Manual load. The player clicks "Load" in the side panel (or on a start screen when the HTA opens). The game reads the .txt file and restores the state completely.

File location:

Same folder as the .hta file. Simple. No complex file paths. The save file sits right next to the game file.

1.12 Technical Foundation

The game is built as a single .hta file. HTA (HTML Application) is a Microsoft technology that runs HTML, CSS, and JavaScript as a standalone desktop application on Windows. It opens its own window. It has its own taskbar icon. It is not a browser tab. It requires zero installation, zero external dependencies, and zero internet connection.

What the agent produces:

One .hta file. That's it. The player double-clicks it, a window opens, the game runs.
The .hta file contains all HTML, CSS, and JavaScript inline. No external files (except the save .txt file it creates).
The code inside must be well-organized. Even though it's one file, the JavaScript should be structured into clear sections: event engine, UI rendering, game state management, save/load system, crew logic, ship logic, station logic, quest logic, etc. Comments should mark each section clearly.

HTA-specific settings:

The HTA should declare its window properties: title, size (a comfortable desktop window, not fullscreen by default), icon if possible.
The HTA should use <meta http-equiv="X-UA-Compatible" content="IE=edge"> to get the best rendering mode available.
Note: HTA uses the Trident (Internet Explorer) rendering engine. This means modern JavaScript features (ES6+) may not be fully available. The agent must write JavaScript compatible with this environment. No arrow functions, no let/const (use var), no template literals, no promises, no modern array methods that IE doesn't support. This is a critical technical constraint.

Code architecture:

The game state should be stored in a single global state object (or a small number of clearly defined objects). This makes saving and loading straightforward — serialize the state to JSON, write to file; read from file, parse JSON, restore state.
The UI should be rendered from the game state. When the state changes, the UI updates to reflect it. This is a simple version of a reactive pattern — not a full framework, just a discipline: state changes -> re-render relevant parts of the screen.
The event engine should be data-driven. Events are defined as objects with properties. The engine reads these objects and executes them based on their type, trigger conditions, countdown values, and chain links. Adding a new event should mean adding a new event object to a list, not writing new engine logic.
Functions should be small and focused. Each function does one thing. The code should be readable by a human (or by the agent in future sessions when expanding the game).

=== END OF MANIFESTO ===